

Programmieren mit R – Grundkurs

Dr.-Ing. Grigory Devadze

Worum es geht

Dieser Kurs führt Sie vom ersten Befehl in R bis zu reproduzierbaren Berichten und KI-gestützten Workflows. Wir arbeiten durchgängig an echten öffentlichen Datensätzen und in einem klaren Arbeitsrhythmus.

Vom Werkzeug zur Routine

RStudio, Projekte und Skripte werden nicht abstrakt gezeigt, sondern als wiederholbarer Arbeitsablauf eingeübt.

Echte Daten, viele Schritte

Wir arbeiten mit **mehreren echten Datensätzen aus deutschen Behörden- und Statistik-Quellen**: Strompreis-/CO₂-Zeitreihe 2024, Wetter Berlin 2024, Bevölkerungs-Zeitreihen deutscher Gemeinden 1980–2024.

Code lesen vor Code tippen

Jede neue Idee zuerst kurz lesen, vorhersagen, dann ausführen und in eigenen Worten erklären.

Vom Befehl zum Bericht

Am Ende stehen eine eigene Analyse, ein sauber gerendertes Quarto-Dokument und ein Ausblick auf KI-Werkzeuge.

 **Zielzustand:** Sie können einen Datensatz laden, prüfen, transformieren, statistisch auswerten, visualisieren und in einem reproduzierbaren Bericht ausliefern.

Agenda

Neun Blöcke und zwei Mini-Übungen:

I

R und RStudio

Sprache, Oberfläche, Pakete.

II

Sprache und Strukturen

Vektoren, Matrizen, Listen, Tibbles.

III

Funktionen und Kontrolllogik

`if / ifelse` , Vektorisierung, `map` .

IV

Datenimport und -export

CSV, Excel, SPSS, DB, RDS.

V

tidyverse

`mutate` , `group_by` , Joins, Pivot.

VI

Statistik

Korrelation, Tests, `lm` , `glm` .

VII

ggplot2

Grammar of Graphics, Facets, Themes.

VIII

Quarto-Berichte

HTML/PDF/Word/Slides, interaktiv.

IX

KI in R

Copilot, `ellmer` , lokale LLMs.

Lernziele

Nach diesem Kurs können die Teilnehmenden:

- R als **Statistik-, Daten- und Reporting-Werkzeug** einordnen und in **RStudio / Posit** sicher zwischen Konsole, Skript, Projekt und Report wechseln
- die Grundtypen **Vektor, Matrix, Liste, Data Frame, Tibble** unterscheiden und mit eigenen **Funktionen** + bewusst gewählter Vektorisierung arbeiten
- Daten aus **CSV, Excel, SPSS/SAS/Stata, Datenbanken und RDS** importieren, prüfen und sauber exportieren
- mit dem **tidyverse** Datensätze transformieren, gruppieren, aggregieren und in lesbare Tabellen überführen
- die wichtigsten **deskriptiven Statistiken, Korrelationsmaße** und **Signifikanztests** in R berechnen und richtig einordnen
- **lineare** und **logistische Regression** rechnen, Diagnostik prüfen und Modelle vergleichen
- saubere **ggplot2-Grafiken** für Balken, Streuung, Linie, Boxplot und Histogramm aufbauen
- **Quarto-Berichte** mit mehreren Output-Formaten und interaktiven Tabellen/Diagrammen aufsetzen
- **KI-Werkzeuge** (Copilot, ellmer, rollama, chattr, gptstudio) in den eigenen R-Workflow einbinden und Datenschutz-Grenzen erkennen

So arbeiten wir

1

Kurz einordnen

Welches Problem löst der neue Begriff im späteren Workflow?

2

Minimalcode lesen

Zuerst die Struktur erkennen, dann erst tippen.

3

Eine Zeile variieren

Filter, Spalte oder Argument gezielt ändern und Ergebnis beobachten.

4

Ergebnis versprachlichen

Was ist fachlich passiert — nicht nur technisch?

5

Muster sichern

Funktionierende Variante im Skript belassen und kurz kommentieren.

Woran man Verständnis erkennt

- vor der Ausführung eine Vorhersage formulieren
- eine Zeile bewusst ändern können
- das Ergebnis in Alltagssprache erklären

Teil I — Einführung in R und RStudio

Sprache, Ökosystem und ein produktiver Arbeitsfluss in RStudio / Posit.

Einordnung

Wann R fachlich besonders stark ist — und wann nicht.

Werkzeuge

RStudio / Posit, **tidyverse** und Quarto als Kernlinie.

Routine

Konsole, Skript, Projekt und Report.

Was ist R?

Sprache

R ist eine **Programmiersprache für Statistik, Datenanalyse und Visualisierung** — aus der akademischen Statistik entstanden, heute auch in Wirtschaft und Verwaltung Standard.

Arbeitsstil

Typisch ist der Weg **Konsole → Skript → Projekt → Report**, nicht eine Kette klick-orientierter Einzelschritte.

Ökosystem

Im Alltag prägen Pakete den Kurs: `dplyr`, `tidyr`, `readr`, `ggplot2`, `readxl`, `haven`, `DBI`, `quarto`.

Zielgruppen

Besonders stark in Statistik, Biostatistik, Ökonometrie, Marktforschung, Ausbildung und reproduzierbarem Reporting.

Wann ist R besonders stark?

Bedarf	Einordnung von R
Klassische Statistik, Inferenz, Modelle	sehr stark
Datenaufbereitung und Wrangling	sehr stark , besonders mit tidyverse
Reproduzierbare Berichte	sehr stark , besonders mit Quarto
Explorative Datenanalyse	stark , schnelle Iteration
Dashboards / Shiny	gut möglich
Deep Learning / große Produktivsysteme	möglich, aber selten die Standardroute



R passt besonders gut, wenn **Analyse und Kommunikation** im Vordergrund stehen. Geht es später stark Richtung **MLOps, Deployment, Software-Engineering im Großen**, ist Python oft die naheliegende Ergänzung.

R oder Python

R zuerst

- Statistik und Inferenz
- wissenschaftliche Auswertungen
- tabellarische Analysen
- schnelle Berichte und Visualisierungen

Python zuerst

- breite ML- und DL-Ökosysteme
- Backends, APIs, Produktivsysteme
- allgemeine Softwareentwicklung

Beides ist sinnvoll

Viele Teams analysieren in **R** und deployen produktionsnahe Komponenten in **Python** — die Sprachen ergänzen sich.

Didaktische Konsequenz

Sprachwahl folgt der konkreten Aufgabe: **welches Problem soll zuerst gelöst werden?** Daraus folgt die Toolwahl, nicht andersherum.

Die Arbeitsumgebung in RStudio / Posit

Source / Editor

Hier entstehen Skripte (`.R`), Quarto-Dateien (`.qmd`) und wiederverwendbarer Code. Das ist die eigentliche Arbeitsfläche.

Console

Zum schnellen Testen, Fehlersuchen und Explorieren — gut für 1–2 Zeilen, **schlecht** als alleinige Arbeitsform.

Environment / History

Zeigt alle Objekte im Speicher und protokolliert eingegebene Befehle. Wichtig beim Nachvollziehen einer Sitzung.

Files / Plots / Packages / Help

Dateien, Diagrammvorschau, Paketverwaltung und Dokumentation an einem Ort — alle vier öfter benutzen als Anfänger erwarten.

Konsole, Skript, Projekt, Report

Schritt 1 — Konsole

Zum Ausprobieren. Gut für eine Zeile. Schlecht als alleinige Arbeitsform, weil Schritte verloren gehen.

Schritt 2 — Skript

Hält wiederholbare Schritte fest: laden, transformieren, prüfen, plotten. Die Basis professioneller Arbeit.

Schritt 3 — Projekt

Bündelt Skripte, Daten und Outputs in einem Kontext. Relative Pfade werden dadurch wieder zuverlässig.

Schritt 4 — Report

Quarto verbindet Code, Ergebnis und Erklärung zu einem reproduzierbaren Artefakt — kommt morgen ausführlich.

Projekte, Ordnerstruktur und relative Pfade

```
1 projekt-r-einstieg/  
2 |— projekt-r-einstieg.Rproj  
3 |— data/  
4 |   |— strompreise.csv  
5 |   |— berlin_weather_2024.csv  
6 |   |— bevoelkerung_zeitreihe.xlsx  
7 |— scripts/  
8 |   |— 01_import.R  
9 |   |— 02_analyse.R  
10 |— output/  
11 |   |— kennzahlen.csv  
12 |— report/  
13 |   |— bericht.qmd
```

- Pro Thema **ein Projekt** anlegen — über `File` → `New Project`.
- Rohdaten in `data/` getrennt von Outputs in `output/`.
- **Immer relative Pfade** verwenden: `"data/strompreise.csv"`, nie `"C:/Users/..."`.
- Skripte durchnummerieren, damit die Reihenfolge klar bleibt.
- Ein Projekt ist die kleinste Einheit **reproduzierbarer** R-Arbeit.

Pakete installieren, laden und Hilfe finden

```
1 # Einmalig auf dem Rechner installieren
2 install.packages("tidyverse")
3 install.packages("readxl")
4 install.packages("haven")
5
6 # Pro R-Sitzung laden
7 library(tidyverse)
8 library(readxl)
9
10 # Hilfe und Beispiele
11 ?read_csv
12 help("summarise")
13 example(mean)
14 vignette("dplyr")
```

- **Installieren** heißt: Paket einmalig auf den Rechner holen.
- **Laden** mit `library()` heißt: in der aktuellen Sitzung verfügbar machen.
- `?funktion` öffnet die Hilfeseite, `example()` zeigt lauffähige Mini-Beispiele.
- `vignette()` liefert ausführliche Tutorials.
- Gute R-Arbeit heißt auch: **Dokumentation lesen können**.

Eine erste Arbeitssitzung in R

1

Projekt öffnen

`.Rproj` doppelklicken — dadurch sind Pfade automatisch korrekt.

2

Skript anlegen

`File → New File → R Script`. Nicht in der Console verlieren.

3

Pakete laden

Nur das, was die konkrete Aufgabe wirklich braucht.

4

Daten importieren

`read_csv("data/strompreise.csv")` oder
`read_excel("data/bevoelkerung_zeitreihe.xlsx")`.

5

Kurz prüfen

`glimpse()`, `summary()`,
`count()` für sofortige Orientierung.

Erster echter Erfolg

Regionen zählen, einen Filter setzen, eine erste Kennzahl berechnen — schon ist es ein Workflow, kein Klick.

Teil II — Sprache, Objekte, Datenstrukturen

Die Bausteine, mit denen R Werte und Tabellen im Speicher hält.

Werte und Typen

Wie R Werte speichert und warum Typen spätere Fehler erklären.

Container

Vektor, Matrix, Liste, Data Frame, Tibble — wann welcher Container.

Vektoriellles Denken

Indexing, Masken und Operationen über ganze Spalten statt zeilenweise.

Zuweisung, Namen und Objekte

```
1 x <- 10
2 preis_netto <- 99.90
3 name <- "Regionalverband"
4 aktiv <- TRUE
5
6 x <- x + 5
7 print(x)          # 15
8
9 ls()              # vorhandene Objekte
10 rm(aktiv)         # gezielt entfernen
11 rm(list = ls())   # alles löschen (mit Bedacht)
```

- `<-` weist einem Namen einen Wert zu. `=` funktioniert auch, ist aber unüblich.
- R ist **case-sensitive**: `wert` und `Wert` sind verschieden.
- Namen sollten sprechend und in Kleinbuchstaben sein: `umsatz_2025` schlägt `u1`.
- `ls()` zeigt das aktuelle Environment, `rm()` räumt auf.
- Beim Kürzel `Alt+-` setzt RStudio den Pfeil automatisch.

Grundlegende Datentypen

```
1 num      <- 7
2 anzahl1  <- 7L
3 name     <- "Anna"
4 flag     <- FALSE
5 heute    <- as.Date("2026-05-25")
6 seg      <- factor(c("B2B", "B2C", "B2B"))
7
8 typeof(num)
9 is.integer(anzahl1)
10 as.character(num)
11 as.numeric("42")
12 levels(seg)
```

Häufige Typen

- `numeric` / `double` — Kommazahlen
- `integer` — Ganzzahlen (`7L`)
- `character` — Text in `" "`
- `logical` — `TRUE` / `FALSE`
- `factor` — kategoriale Werte mit Levels
- `Date` , `POSIXct` — Datum und Zeitstempel

 Falsche Typen sind die häufigste Ursache stiller Fehler — besonders nach CSV-Import.

Vektoren sind der Grundbaustein

```
1 numbers <- c(1, 2, 3, 4, 5)
2 namen   <- c("Anna", "Bernd", "Clara")
3 flags   <- c(TRUE, FALSE, TRUE)
4
5 seq_vec <- 1:10           # 1,2,...,10
6 rep_vec <- rep(0, 4)      # 0,0,0,0
7 seq(0, 1, by = 0.25)     # 0,0.25,0.5,0.75,1
8
9 numbers * 10              # vektoriell
10 numbers[numbers > 3]     # Maske
11 mean(numbers); length(numbers)
```

- Ein Vektor ist eine **geordnete Sammlung gleichartiger Werte**.
- Operationen wirken **elementweise**: `numbers * 10` multipliziert jedes Element.
- Bei gemischten Typen **coerct** R automatisch: `c(1, "a")` wird **character**.
- Coercion-Reihenfolge: **logical** < **integer** < **double** < **character**.
- Vektoriell Denken ist das wichtigste Idiom in R.

Matrizen

```
1 m <- matrix(1:12, nrow = 3, ncol = 4)
2 m
3 #      [,1] [,2] [,3] [,4]
4 # [1,]    1    4    7   10
5 # [2,]    2    5    8   11
6 # [3,]    3    6    9   12
7
8 dim(m)           # 3 4
9 t(m)             # transponieren
10 m[2, 3]          # Zeile 2, Spalte 3
11 m[, 2]           # gesamte 2. Spalte
12
13 a <- matrix(1:4, 2, 2)
14 b <- matrix(5:8, 2, 2)
15 a %*% b          # Matrixmultiplikation
16 rbind(a, b); cbind(a, b)
```

- Eine **Matrix** ist ein 2D-Vektor: alle Elemente desselben Typs, mit `dim()`-Attribut.
- `matrix(daten, nrow, ncol)` baut sie; standardmäßig spaltenweise gefüllt.
- `%*%` ist Matrixmultiplikation, `*` bleibt elementweise.
- `rbind()/cbind()` setzt Matrizen zeilen-/spaltenweise zusammen.
- In der Datenanalyse meist Übergang zu Data Frames — Matrizen bleiben wichtig für lineare Algebra und Statistik.

Listen

```
1 ergebnis <- list(  
2   titel      = "Strompreis-Analyse 2024",  
3   laender    = c("Hessen", "Bayern", "Berlin"),  
4   kennzahlen = c(mittel_preis = 0.087, max_preis = 0.612),  
5   modell     = lm(mpg ~ wt, data = mtcars)  
6 )  
7  
8 ergebnis$titel      # per Name  
9 ergebnis[["laender"]] # ein Element  
10 ergebnis["laender"]  # weiterhin eine Liste  
11 names(ergebnis)  
12 length(ergebnis)
```

- Eine **Liste** darf Elemente **unterschiedlicher Typen** und Längen enthalten.
- Zugriff mit **\$name** oder **[[]]** liefert das **Element selbst**.
- **[]** liefert weiterhin eine (Teil-)Liste — wichtige Unterscheidung.
- Modelle (**lm**, **glm**) und viele Funktionsergebnisse sind intern Listen.
- Wer Listen versteht, versteht später **purrr::map_*()** schneller.

Indexing und logische Masken

```
1 numbers <- c(10, 20, 30, 40, 50)
2
3 numbers[2]           # 20
4 numbers[c(1, 4)]     # 10 40
5 numbers[-3]          # 10 20 40 50
6
7 mask <- numbers >= 30
8 numbers[mask]         # 30 40 50
9 numbers[numbers >= 30] # gleiches Ergebnis
10
11 named <- c(Hessen = 6.3, Bayern = 13.1, Berlin = 3.7)
12 named[["Bayern"]]    # 13.1
13 named["Bayern"]      # benannt: Bayern = 13.1
```

- `[ ]` wählt Teilbereiche.
- **Negative Indizes** schließen Elemente aus.
- Logische Vektoren filtern direkt — das ist das wichtigste Idiom.
- `[[ ]]` holt **genau ein Element** ohne Umhüllung.
- Dasselbe Muster wiederholt sich später bei Listen, Spalten und Data Frames.

Data Frames und Tibbles

```
1 library(tibble)
2
3 bev <- tibble(
4   gkz      = c("06411", "06412", "06413"),
5   name     = c("Darmstadt", "Frankfurt", "Offenbach"),
6   jahr     = c(2024, 2024, 2024),
7   einwohner = c(159207, 773068, 130892),
8   land     = c("Hessen", "Hessen", "Hessen")
9 )
10
11 bev$name           # eine Spalte
12 bev[["name"]]      # gleich
13 bev[1, ]           # erste Zeile
14 nrow(bev); ncol(bev)
```

- Ein **Data Frame** ist eine Tabelle: jede **Spalte** ein Vektor, alle gleich lang.
- Spalten können **unterschiedliche Typen** haben — anders als bei Matrizen.
- Ein **Tibble** ist die **tidyverse**-Variante: schöner Print, keine Auto-Konvertierung von Strings zu Factors, strenger bei `[[ ]]`.
- Für moderne Datenarbeit ist der Schritt **Vektor → Tabelle** zentral.
- `data.frame()` und `tibble()` sind im Alltag austauschbar; Tibbles sind oft angenehmer.

glimpse, summary, head und NAs

```
1 library(dplyr)
2 library(tidyr)
3
4 head(bev, 5)
5 tail(bev, 3)
6 glimpse(bev)           # Spalten + Typen
7 str(bev)               # Basis-Variante
8 summary(bev)
9
10 # NA-Diagnose
11 sum(is.na(bev$einwohner))
12 mean(bev$einwohner, na.rm = TRUE)
13
14 bev |>
15   drop_na(einwohner)
```

- **Immer früh prüfen:** Spaltenzahl, Typen, Wertebereiche, fehlende Werte.
- `glimpse()` ist der schnellste Überblick — kompakt und vollständig.
- Fehlende Werte heißen **NA**. Viele Funktionen brauchen `na.rm = TRUE`.
- `is.na(x)` liefert eine logische Maske — kombinierbar mit `sum()` und `mean()`.
- Ein Datensatz ist erst nach einer Plausibilitätsprüfung "wirklich gelesen".

Teil III — Funktionen und Kontrolllogik

Eigene Funktionen, Bedingungen und das Mapping-Denken in R.

Wiederverwenden

Aus Code-Schnipseln werden kleine, benannte Funktionen.

Entscheiden

`if, else if, else` und vektorielles `ifelse()`.

Wiederholen

Schleife oder Vektorisierung — und wann `lapply/map` der elegantere Weg ist.

Häufige Basisfunktionen

```
1 werte <- c(8, 12, 5, 14, 9, 11, 7)
2
3 sum(werte);   mean(werte);   median(werte)
4 sd(werte);    var(werte)
5 min(werte);   max(werte);    range(werte)
6 round(3.14159, 2)
7 sort(werte);  rev(werte)
8 unique(c(1, 1, 2, 3, 3, 3))
9 length(werte)
10
11 laender <- c("Hessen", "Bayern", "Hessen",
12             "Berlin", "Bayern", "Hessen")
13 table(laender)      # Häufigkeitstabelle
```

- Man muss nicht hunderte Funktionen kennen — sondern die **Logik**: Objekt rein, Ergebnis raus.
- `sum`, `mean`, `median`, `sd`, `min`, `max` decken den Alltag der deskriptiven Statistik ab.
- `unique()` und `table()` sind unverzichtbar bei kategorialen Variablen.
- `round()` zur sauberen Anzeige, **nicht** zur Manipulation der Daten.
- Hilfe immer einen `?`-Tastendruck entfernt.

Eigene Funktionen schreiben

```
1 rabatt_preis <- function(preis, rabatt = 0.10) {  
2   preis * (1 - rabatt)  
3 }  
4  
5 rabatt_preis(100)           # 90  
6 rabatt_preis(100, rabatt = 0.2) # 80  
7  
8 roi <- function(umsatz, kosten) {  
9   if (kosten <= 0) return(NA_real_)  
10  umsatz / kosten  
11 }  
12  
13 roi(umsatz = 50000, kosten = 4000) # 12.5  
14 roi(50000, 0)                     # NA
```

- `function(arg1, arg2 = default)` definiert Argumente und Standardwerte.
- Die **letzte ausgewertete Zeile** ist der Rückgabewert — `return()` ist optional, aber bei Sonderfällen nützlich.
- Sprechende Argumentnamen verbessern die Lesbarkeit beim Aufruf.
- Faustregel: Sobald Sie denselben Code dreimal kopieren, gehört er in eine Funktion.
- Kleine Funktionen sind die Voraussetzung für `lapply`, `sapply`, `map_*`.

Kontrolllogik: `if` und vektorielles `ifelse()`

```
1 # Einzelentscheidung
2 note <- 74
3
4 if (note >= 90) {
5   kategorie <- "sehr gut"
6 } else if (note >= 75) {
7   kategorie <- "gut"
8 } else {
9   kategorie <- "noch ausbaufähig"
10 }
11
12 # Vektoriell: für ganze Spalten geeignet
13 noten <- c(50, 78, 95, 88, 62)
14 ifelse(noten >= 75, "bestanden", "offen")
15 # "offen"      "bestanden" "bestanden"
16 # "bestanden" "offen"
```

- `if` arbeitet auf **einem** Wert — für skalare Entscheidungen.
- `ifelse()` arbeitet **elementweise** und ist für Spalten passend.
- Für mehrstufige Datenlogik kommt später `dplyr::case_when()` — viel lesbarer als verschachtelte `ifelse`.
- Faustregel: in Funktionen `if`, in Daten-Pipelines `case_when`.

Schleifen vs. Vektorisierung

```
1 # Schleife – funktioniert, aber umständlich
2 werte <- c(10, 20, 30, 40)
3 mit_steuer <- numeric(length(werte))
4
5 for (i in seq_along(werte)) {
6   mit_steuer[i] <- werte[i] * 1.19
7 }
8 mit_steuer
```

```
1 # Vektoriell – idiomatisches R
2 werte <- c(10, 20, 30, 40)
3 mit_steuer <- werte * 1.19
4 mit_steuer
```

 R **bevorzugt vektorielle Operationen**. Schleifen sind nicht verboten, aber bei tabellarischen Daten meist umständlich und langsamer.

lapply, sapply und purrr::map_*

```
1 spalten <- list(  
2   strompreis = c(0.082, 0.107, 0.061),  
3   co2_faktor = c(174.4, 198.2, 142.9),  
4   emissionen = c(9980.5, 11250.7, 8203.4)  
5 )  
6  
7 lapply(spalten, mean)           # Liste  
8 sapply(spalten, mean)          # benannter Vektor  
9  
10 purrr::map(spalten, mean)       # immer Liste  
11 purrr::map_dbl(spalten, mean)   # immer double-Vektor  
12 purrr::map_dbl(spalten, ~ mean(.x, na.rm = TRUE))
```

- **Mapping-Idee:** “wende dieselbe Funktion auf jedes Element an”.
- `lapply()` liefert eine Liste, `sapply()` versucht zu vereinfachen.
- `purrr::map_*()` ist die moderne **tidyverse**-Variante mit **vorhersagbarem Rückgabety**p (`_dbl`, `_chr`, `_lgl`, `_int`).
- Über `~ funktion(.x)` lassen sich kurze Lambdas formulieren.
- Dieses Muster wiederholt sich überall, wo “viele Spalten gleich behandeln” gefragt ist.

Teil IV — Datenimport und Datenexport

Dateien lesen, prüfen und sauber wieder speichern.

Formate

CSV, Excel, SPSS/SAS/Stata, Datenbanken und RDS bewusst einordnen.

Qualität

Zeilen, Spalten, Typen und **NAs** sofort prüfen.

Export

Ergebnisse reproduzierbar in den richtigen Ordner schreiben.

Typische Datenquellen in R

CSV / Textdateien

Häufigster Startpunkt. Klein, portabel, mit jedem Tool offenbar.

`readr::read_csv()` ist die moderne Standardroute.

Excel

Sehr verbreitet in Fachabteilungen — oft mit mehreren Sheets, verbundenen Zellen und manuellen Formatierungen. Paket:

`readxl`.

SPSS / SAS / Stata

Relevant in Sozialwissenschaften, Verwaltung, klinischer Forschung. Paket: `haven` (modern) oder `foreign` (klassisch).

Datenbanken, APIs, RDS

Für wiederkehrende Workflows. `DBI` + Treiberpakete (`odbc` , `RSQLite` , `RPostgres`), `httr2` für APIs, `saveRDS()` / `readRDS()` für R-interne Zwischenspeicher.

 Import ist nie nur Technik. Entscheidend ist, **welche Struktur und welche Datentypen nach dem Laden wirklich vorliegen** — das zeigt erst `glimpse()`.

Beispieldatensätze

Wir arbeiten durchgängig mit fünf echten Datensätzen aus deutschen Behörden- und Statistik-Quellen:

`data/strompreise.csv`

8 784 stündliche Beobachtungen für 2024: **Zeitstempel**, **CO₂-Faktor** des Strommix, **Strompreis** (EUR/kWh), **absolute Emissionen**. Kernquelle für Zeitreihen, Korrelation, Regression.

`data/berlin_weather_2024.csv`

8 784 stündliche Beobachtungen Berlin 2024: `time`, `temperature_2m`, `precipitation`, `sunshine_duration`. Zweite Zeitreihe — gut für Joins über die Stundenmarke.

`data/bevoelkerung_zeitreihe.xlsx`

109 Gemeinden × 45 Jahre (1980–2024) im **wide format**: erste Spalte `gkz`, dann pro Jahr eine Einwohnerspalte. Beispiel für Excel-Import und `pivot_longer()`.

`data/schluesselfabelle.xlsx`

Lookup über `gkz`: `name`, `gebiet_typ`, `land`, Kreis- und Verdichtungsraum-Zuordnung. Klassische **Stammdatentabelle** für Joins.

`data/auslaender_absolut.xlsx`

Parallele Struktur zur Bevölkerungs-Zeitreihe — zum Üben weiterer Pivot- und Join-Aufgaben.

CSV lesen und sofort prüfen

```
1 library(readr)
2 library(dplyr)
3 library(lubridate)
4
5 strompreise <- read_csv(
6   "data/strompreise.csv",
7   show_col_types = FALSE,
8   name_repair = "unique"
9 ) |>
10  rename(
11    zeit = `date_id...1`,
12    co2_faktor = `CO2-Emissionsfaktor des Strommix`,
13    strompreis = Strompreis,
14    emissionen = `Absolute Emissionen`
15 ) |>
16  select(zeit, co2_faktor, strompreis, emissionen) |>
17  mutate(zeit = as_datetime(zeit))
18
19 glimpse(strompreise)
20 nrow(strompreise); ncol(strompreise)
21 summary(strompreise$strompreis)
22 sum(is.na(strompreise))
```

- `read_csv()` aus `readr` ist die moderne Standardroute (schneller, vorhersagbarer als `read.csv()`).
- Direkt nach dem Laden: **Dimensionen, Spaltennamen, Typen, NAs** prüfen.
- Typische Fallstricke: Trennzeichen (, vs. ;), Dezimalzeichen (. vs. ,), Encoding (UTF-8 vs. Windows-1252), Datumsformat.
- Für deutsche Excel-Exporte mit ; und , -Dezimal: `read_csv2()`.
- `locale = locale(decimal_mark = ",", grouping_mark = ".")` für individuelle Konfiguration.

Excel-Dateien mit `readxl`

```
1 library(readxl)
2
3 # Was steckt in der Datei?
4 excel_sheets("data/bevoelkerung_zeitreihe.xlsx")
5 # [1] "Sheet1"
6
7 bev <- read_excel(
8   "data/bevoelkerung_zeitreihe.xlsx",
9   sheet = 1
10 )
11
12 # Stammdaten-Tabelle mit Typen
13 schluessel <- read_excel(
14   "data/schluesseltabelle.xlsx",
15   sheet = 1,
16   col_types = c("text", "text", "text", "text",
17                 "text", "text", "text", "text")
18 )
```

- `readxl` braucht **kein** Excel auf dem Rechner — pure R-Implementierung.
- `excel_sheets()` zeigt alle Tabellenblätter, `sheet =` wählt eines.
- `range =` schneidet einen Zellbereich heraus, `col_types =` erzwingt Typen.
- `skip =` überspringt Kopfzeilen oder Logo-Zeilen.
- Für riesige `.xlsx` oder zum Schreiben: `openxlsx2`, `writexl`.

SPSS, SAS und Stata mit **haven**

```
1 library(haven)
2
3 # SPSS
4 umfrage <- read_sav("daten/umfrage_2025.sav")
5 glimpse(umfrage)
6
7 # Stata
8 panel <- read_dta("daten/panel.dta")
9
10 # SAS
11 ergebnis <- read_sas("daten/output.sas7bdat")
12
13 # Wertelabels weiter nutzen
14 labelled::var_label(umfrage$bildung)
15 as_factor(umfrage$bildung) # Labels -> Factor
16
17 # Klassische Alternative
18 foreign::read.spss("daten/umfrage_2025.sav",
19                   to.data.frame = TRUE)
```

- **haven** ist die moderne **tidyverse**-Variante: liest **.sav**, **.dta**, **.sas7bdat**, **.xpt**.
- **Variable Labels und Value Labels** bleiben erhalten — wichtig für SPSS-Workflows.
- **as_factor()** wandelt SPSS-Labels in echte R-Factors.
- **foreign** (Basis-R) ist Alternative — kennt weniger Formate, kann aber im Notfall einspringen.
- Für SPSS-Syntax-Migration oft hilfreich: **sjlabelled**.

Datenbanken mit DBI + Treiber

```
1 library(DBI); library(RSQLite)
2 con <- dbConnect(SQLite(), "data/energie.sqlite")
3
4 dbListTables(con)
5 dbListFields(con, "strompreise")
6
7 # Ganze Tabelle
8 db_tab <- dbReadTable(con, "strompreise")
9
10 # Gezielte SQL-Abfrage
11 preise <- dbGetQuery(con, "
12     SELECT date(zeit) AS tag,
13            AVG(strompreis) AS preis_mittel
14     FROM strompreise
15     GROUP BY tag
16     ORDER BY preis_mittel DESC")
17
18 dbWriteTable(con, "tagespreise", preise)
19 dbDisconnect(con)
```

- DBI ist die einheitliche Schnittstelle, die Treiberpakete (*RSQLite*, *odbc*, *RPostgres*, *RMariaDB*) liefern die Verbindung.
- `dbConnect()` öffnet, `dbDisconnect()` schließt sauber.
- `dbGetQuery()` schickt SQL ab und holt das Ergebnis als Data Frame.
- `dbWriteTable()` schreibt zurück — z. B. eine Ergebnistabelle.
- `dbplyr` erlaubt sogar *tidyverse*-Syntax gegen die Datenbank, ohne SQL zu schreiben.

rio: ein Befehl für viele Formate

```
1 library(rio)
2
3 # Format wird aus der Endung erkannt
4 csv_daten <- import("data/strompreise.csv")
5 xlsx_daten <- import("data/bevoelkerung_zeitreihe.xlsx",
6                     sheet = 1)
7 sav_daten <- import("data/umfrage.sav")
8 rds_daten <- import("data/cache.rds")
9
10 # Exportieren analog
11 export(csv_daten, "output/strompreise.csv")
12 export(csv_daten, "output/strompreise.xlsx")
13 export(csv_daten, "output/strompreise.dta")
14
15 # Mehrere Sheets auf einmal schreiben
16 export(list(Stammdaten = schluessel,
17            Kennzahlen = kpi),
18        "output/bericht.xlsx")
```

- `rio` bündelt viele Reader/Writer hinter zwei Funktionen: `import()` und `export()`.
- Praktisch für **gemischte Format-Landschaften** und Skripte, die mit verschiedenen Quellen zurechtkommen müssen.
- Didaktisch: erst die spezifischen Reader (`read_csv`, `read_excel`, `read_sav`) verstehen — dann `rio` als bequeme Abkürzung.
- Im Hintergrund werden die "richtigen" Pakete automatisch aufgerufen.

Ergebnisse speichern: CSV, Excel, RDS

```
1 library(readr); library(writexl)
2
3 # CSV – Standardaustausch
4 write_csv(strompreise,
5           "output/strompreise_bereinigt.csv")
6
7 # Excel – für Fachanwender
8 write_xlsx(strompreise,
9            "output/strompreise_bereinigt.xlsx")
10
11 # RDS – R-interner Zwischenstand
12 saveRDS(strompreise,
13         "output/strompreise_bereinigt.rds")
14 strompreise_zurueck <-
15   readRDS("output/strompreise_bereinigt.rds")
16
17 # Mehrere Objekte zusammen
18 save(strompreise, kennzahlen,
19      file = "output/zwischenstand.RData")
20 load("output/zwischenstand.RData")
```

CSV

Universell, aber Typen werden zu Text umgewandelt.

Excel

Für Fachanwender, aber anfällig für manuelle Änderungen.

RDS

Speichert **ein R-Objekt mit allen Typen** — ideal als Zwischenstand.

Faustregel

Roh- und Endformate als CSV/XLSX, **R-interne Zwischenstände** als RDS.

Import- und Export-Checkliste

Struktur

Zeilen- und Spaltenzahl plausibel? Mit `nrow()`, `ncol()`, `dim()` prüfen.

Typen

Sind Zahlen Zahlen, Datum Datum, Codes nicht versehentlich numerisch? `glimpse()` zeigt es.

Fehlende Werte

Unerwartet viele `NA`, leere Schlüssel, doppelte IDs? `summary()` und `count(is.na(x))`.

Namen

Spaltennamen lesbar, stabil, ohne Leerzeichen? Sonst `janitor::clean_names()`.

Kodierung

Umlaute korrekt? Dezimal- und Tausendertrennzeichen wie erwartet?

Reproduzierbarkeit

Wurde die Exportdatei in den richtigen Ordner geschrieben — und lässt sich morgen mit demselben Skript erneut erzeugen?

Teil V — Datenaufbereitung mit dem **tidyverse**

Von tabellarischen Rohdaten zu lesbaren Transformationen und Tabellierungen.

Denkmodell

Tidy Data, Pipe `|>` und kleine lesbare Schritte statt Monster-Code.

Kernverben

`select`, `filter`, `mutate`,
`summarise`, `pivot_*`.

Tabellieren

`count`, `distinct`,
`group_by` + `summarise` für
Kennzahlen.

Tidy Data und die Pipe |>

Tidy Data

- jede **Zeile** = eine Beobachtung
- jede **Spalte** = eine Variable
- jede **Zelle** = ein Wert

Pipe |>

“Nimm Objekt A und reiche es als erstes Argument an den nächsten Schritt weiter.”

```
1 library(dplyr)
2
3 strompreise |>
4   filter(strompreis > 0.15) |>
5   select(zeit, co2_faktor, strompreis) |>
6   arrange(desc(strompreis))
```

 Lesen Sie die Pipe als deutschen Satz: **“Nimm strompreise, filtere Spitzenstunden, wähle drei Spalten, sortiere absteigend”.**

select, filter, arrange

```
1 # Spalten wählen
2 strompreise |>
3   select(zeit, strompreis, co2_faktor)
4
5 strompreise |>
6   select(-emissionen)
7
8 strompreise |>
9   select(zeit, starts_with("strom"))
10
11 # Zeilen filtern
12 strompreise |>
13   filter(strompreis > 0.20, co2_faktor < 300)
14
15 strompreise |>
16   filter(strompreis < 0)           # negative Preise
17
18 # Sortieren
19 strompreise |>
20   arrange(desc(strompreis))
```

select()

Spalten auswählen — auch mit Helfern wie `starts_with()`, `ends_with()`, `where()`.

filter()

Zeilen nach Bedingungen — Komma bedeutet UND, `|` bedeutet ODER.

arrange()

Sortieren, `desc()` für absteigend, mehrere Spalten möglich.

Erste drei Verben

Decken bereits einen großen Teil des Alltags ab.

mutate, case_when, across

```
1 strompreise |>
2   mutate(
3     preis_ct  = strompreis * 100,
4     monat     = lubridate::floor_date(zeit, "month"),
5     stunde     = lubridate::hour(zeit),
6     klasse = case_when(
7       strompreis < 0      ~ "negativ",
8       strompreis < 0.05  ~ "günstig",
9       strompreis < 0.15  ~ "normal",
10      TRUE               ~ "Spitze"
11    )
12  ) |>
13  mutate(across(c(strompreis, co2_faktor, emissionen), round, 2)) |>
14  select(zeit, monat, stunde, strompreis, preis_ct, klasse)
```

 `mutate()` erweitert die Tabelle um neue Spalten. `case_when()` ersetzt verschachtelte `ifelse`. `across()` wendet dieselbe Funktion auf mehrere Spalten an — das ist `*apply` im tidyverse-Stil.

count() und distinct() für Tabellierungen

```
1 schluesel |> count(land, sort = TRUE)
2 #   land      n
3 # 1 Hessen    25
4 # 2 Bayern    18
5 # 3 Berlin     1
6 # ...
7
8 # Häufigkeit nach Kreuzkombination, sortiert
9 schluesel |>
10   count(land, gebiet_typ, sort = TRUE)
11
12 # Eindeutige Werte
13 schluesel |> distinct(land)
14 schluesel |> distinct(land, gebiet_typ)
15
16 # Wie viele Stunden enthält die Strompreis-Tabelle?
17 n_distinct(strompreise$zeit)
```

- `count()` ist der Schnellweg für **Häufigkeitstabellen** — viel lesbarer als `table()`.
- `count(x, y)` liefert eine **Kreuztabulierung** als langes Format.
- `distinct()` reduziert auf eindeutige Kombinationen — gut zur Plausibilitätsprüfung.
- `n_distinct()` zählt eindeutige Werte ohne Gruppierung.
- Diese vier Funktionen reichen für 80 % aller "Beschreiben Sie den Datensatz"-Aufgaben.

group_by() + summarise() → Kennzahlen

```
1 strompreise |>
2   mutate(monat = lubridate::floor_date(zeit, "month")) |>
3   group_by(monat) |>
4   summarise(
5     n_stunden      = n(),
6     avg_strompreis = mean(strompreis),
7     max_strompreis = max(strompreis),
8     avg_co2_faktor = mean(co2_faktor),
9     sd_strompreis  = sd(strompreis),
10    .groups        = "drop"
11  ) |>
12  arrange(desc(avg_strompreis))
```

Idee

Tabelle in Gruppen zerlegen, je Gruppe Kennzahlen berechnen.

n() und n_distinct()

Anzahl Zeilen und eindeutige Werte pro Gruppe.

Mehrere Kennzahlen

Beliebig viele in einem `summarise()`-Aufruf.

.groups = "drop"

Gruppierung explizit auflösen — sonst bleibt sie für nächste Schritte aktiv.

Joins: Tabellen verknüpfen

```
1 # Bevölkerung lang + Stammdaten zusammenführen
2 bev_lang <- read_excel(
3   "data/bevoelkerung_zeitreihe.xlsx", sheet = 1)
4 schluessel <- read_excel(
5   "data/schluesseltabelle.xlsx", sheet = 1)
6
7 # Alle Zeilen aus bev_lang, Stammdaten ergänzt
8 bev_lang |>
9   left_join(schluessel, by = "gkz")
10
11 # Nur Treffer in beiden Tabellen
12 bev_lang |>
13   inner_join(schluessel, by = "gkz")
14
15 # Welche gkz in bev_lang fehlen in der Schluesseltabelle?
16 bev_lang |>
17   anti_join(schluessel, by = "gkz")
```

left_join

Alle Zeilen links,
passende Treffer rechts
ergänzen. Der
Standard-Join im Alltag.

inner_join

Nur Zeilen, für die beide
Seiten Treffer haben.

anti_join

Zeilen links, die rechts
keinen Treffer haben —
gut für Qualitätschecks.

Schlüssel

`by =` explizit setzen;
bei mehreren Spalten:
`by = c("a", "b")`.

Pivoting: `pivot_longer` und `pivot_wider`

```
1 library(tidyr); library(readxl)
2
3 bev_breit <- read_excel(
4   "data/bevoelkerung_zeitreihe.xlsx", sheet = 1)
5
6 # Breit -> lang: Zeile je Gemeinde x Jahr
7 bev_lang <- bev_breit |>
8   pivot_longer(
9     cols      = starts_with("Bevoelkerung_Zeitreihe_"),
10    names_to   = "jahr",
11    values_to  = "einwohner",
12    names_prefix = "Bevoelkerung_Zeitreihe_") |>
13    mutate(jahr = as.integer(substr(jahr, 1, 4)))
14
15 # Lang -> breit (zurück in Excel-Form)
16 bev_lang |>
17   pivot_wider(names_from = jahr,
18               values_from = einwohner)
```

- **Breit** = eine Spalte pro Variante (gut für Excel-Augen).
- **Lang** = eine Zeile pro Beobachtung (gut für Analyse, Plot, Modell).
- `pivot_longer()` zieht viele Spalten in zwei: **Name** und **Wert**.
- `pivot_wider()` spreizt eine Wert-Spalte auf nach einer Schlüssel-Spalte.
- Faustregel: für Analyse und Plot meist **lang**, für Präsentation manchmal **breit**.

Datensätze beschreiben mit Tabellen

```
1 library(knitr); library(gt)
2
3 kennzahlen <- strompreise |>
4   mutate(monat = lubridate::floor_date(zeit, "month")) |>
5   group_by(monat) |>
6   summarise(
7     avg_strompreis = mean(strompreis),
8     avg_co2_faktor = mean(co2_faktor),
9     sum_emissionen = sum(emissionen),
10    .groups = "drop"
11  )
12
13 # Schnell und einfach
14 kable(kennzahlen, digits = 3)
15
16 # Präsentationsreif
17 kennzahlen |> gt() |>
18   fmt_number(c(avg_strompreis, avg_co2_faktor), decimals = 3)
```

`knitr::kable()`

Einzeiler für Markdown/Quarto-Reports.

`gt::gt()`

Voll gestaltbare Tabellen mit Formatierung und Gruppen.

Alternativen

`flextable`, `kableExtra`, `reactable` — morgen Thema.

Faustregel

Erst rechnen mit `summarise()`, dann formatieren.

Von Rohdaten zu Kennzahlen

```
1 library(tidyverse); library(lubridate)
2
3 strom_kennzahlen <- read_csv("data/strompreise.csv",
4   show_col_types = FALSE, name_repair = "unique") |>
5   rename(zeit = `date_id...1`,
6     co2_faktor = `CO2-Emissionsfaktor des Strommix`,
7     strompreis = Strompreis,
8     emissionen = `Absolute Emissionen`) |>
9   mutate(zeit = as_datetime(zeit),
10     monat = floor_date(zeit, "month")) |>
11   filter(zeit >= as_datetime("2024-07-01")) |>
12   group_by(monat) |>
13   summarise(avg_strompreis = mean(strompreis),
14     avg_co2_faktor = mean(co2_faktor),
15     sum_emissionen = sum(emissionen),
16     .groups = "drop") |>
17   arrange(monat)
18
19 write_csv(strom_kennzahlen,
20   "output/strom_h2_kennzahlen.csv")
```

Schritt 1 — Einlesen + Typen

CSV laden und Monat als Datum interpretieren.

Schritt 2 — Eingrenzen

Auf das 2. Halbjahr 2024 filtern.

Schritt 3 — Aggregieren

Monatsmittel von Strompreis und CO₂-Faktor, Summe der Emissionen.

Schritt 4 — Sichern

Ergebnistabelle als CSV in `output/` — reproduzierbar.

Mini-Übung 1: Datenaufbereitung

Datei: `labs/lab1_datenaufbereitung.R` — bearbeiten Sie in 25 Minuten:

Auftrag in fünf Schritten

1. Legen Sie ein neues R-Skript an und laden Sie `tidyverse` und `lubridate`.
2. Lesen Sie `data/strompreise.csv` mit `read_csv()` ein, benennen Sie die Spalten in `zeit`, `co2_faktor`, `strompreis`, `emissionen` um und prüfen Sie mit `glimpse()` und `summary()`.
3. Ergänzen Sie eine Spalte `monat = floor_date(zeit, "month")`.
4. Berechnen Sie pro `monat`: Mittelwert `strompreis`, Mittelwert `co2_faktor`, Summe `emissionen`.
5. Speichern Sie die Ergebnistabelle als `output/monats_kennzahlen.csv`.

Erwartetes Ergebnis

Eine Tabelle mit **zwölf Zeilen** (ein Monat pro Zeile) und drei Kennzahlen-Spalten — reproduzierbar erzeugt aus dem Skript.

Zusatzfragen

- In welchem Monat lag der mittlere Strompreis am höchsten?
- Wie viele Stunden mit negativem Strompreis gab es?
- Gibt es `NA`s in `co2_faktor`?

Teil VI — Statistische Verfahren in R

R wurde als Statistiksprache geboren — viele Verfahren stecken ohne Zusatzpaket schon im Basissystem.

Deskriptiv

Lage, Streuung, Quantile und Mehr-auf-einen-Blick mit `summary()` und `skim()`.

Inferenz

Korrelation, klassische Tests und die saubere Interpretation von p-Werten.

Modelle

Lineare und logistische Regression mit Diagnostik und Vergleich.

Deskriptive Statistik: Lage und Streuung

```
1 werte <- c(102, 95, 110, 130, 88, 97, 145, 99, 101, 108)
2
3 mean(werte)          # arithmetisches Mittel
4 median(werte)        # Median
5 sd(werte)            # Standardabweichung
6 var(werte)          # Varianz
7 IQR(werte)           # Interquartilsabstand
8 quantile(werte, probs = c(0.25, 0.5, 0.75))
9 range(werte)         # Min / Max
```

- **Lage** beantwortet "Wo liegt der typische Wert?"
- **Streuung** beantwortet "Wie weit variieren die Werte?"
- `sd()` ist robust gegenüber Vorzeichen, aber sensibel gegenüber Ausreißern.
- `IQR()` ist robuster als `sd()`, wenn Ausreißer im Spiel sind.
- Bei schiefen Verteilungen ist der **Median** ehrlicher als der **Mittelwert**.

Schneller Überblick: `summary`, `psych`, `skimr`

```
1 summary(strompreise)      # Basis-R: Min/Median/Mean/Max
2
3 # psych: klassische Statistik-Übersicht
4 psych::describe(strompreise)
5
6 # skimr: tidyverse-freundlich, sehr lesbar
7 skimr::skim(strompreise)
```

`summary()`

Immer da, oft genug. Bei Faktoren gibt es Häufigkeiten.

`psych::describe()`

Liefert zusätzlich Schiefe, Kurtosis und Standardfehler.

`skimr::skim()`

Beste Wahl für die erste Exploration: getrennt nach Typ, mit Mini-Histogrammen.

Korrelation: `cor()` und `cor.test()`

```
1 # Punktschätzer (drei Methoden)
2 cor(strompreise$co2_faktor, strompreise$strompreis,
3     method = "pearson")
4 cor(strompreise$co2_faktor, strompreise$strompreis,
5     method = "spearman")
6
7 # Inferenz: p-Wert + Konfidenzintervall
8 cor.test(strompreise$co2_faktor, strompreise$strompreis,
9          method = "pearson")
10
11 # Korrelationsmatrix für mehrere Spalten
12 cor(strompreise[, c("strompreis", "co2_faktor",
13                    "emissionen")])
```

- **Pearson**: linearer Zusammenhang, setzt annähernde Normalverteilung voraus.
- **Spearman**: rangbasiert, robust gegenüber Ausreißern und monotonen Nichtlinearitäten.
- **Kendall**: kleiner Stichprobe, sehr robust, schwerer zu interpretieren.
- `cor.test()` liefert **p-Wert + 95%-KI** — beides ist wichtiger als der reine Korrelationswert.

 Hohe Korrelation ist **keine Kausalität**. Immer fachlich plausibilisieren und Konfundierer mitdenken.

Signifikanztests: `t.test()`

```
1 # One-Sample: weicht der mittlere Strompreis von 0.10 EUR ab?
2 t.test(strompreise$strompreis, mu = 0.10)
3
4 # Two-Sample: Tag (6-18 Uhr) vs. Nacht
5 strom_tn <- strompreise |>
6   mutate(phase = ifelse(
7     lubridate::hour(zeit) >= 6 &
8     lubridate::hour(zeit) < 18, "Tag", "Nacht"))
9
10 t.test(strompreis ~ phase, data = strom_tn,
11        var.equal = FALSE) # Welch-Test (Default)
```

One-Sample

Vergleicht den Mittelwert gegen einen Referenzwert (`mu`).

Two-Sample

Vergleicht zwei Gruppen. Welch (Default) ist robuster als der klassische t-Test.

Paired

Für **gepaarte** Beobachtungen (gleiche Einheit, zwei Messzeitpunkte).

Nichtparametrische Alternative: `wilcox.test()`

```
1 # Wenn Normalverteilungs-Annahme zweifelhaft ist
2 wilcox.test(strompreis ~ phase,
3             data = strom_tn)
4
5 # Paired-Variante
6 wilcox.test(vorher, nachher, paired = TRUE)
```

- Vergleicht **Ränge** statt Mittelwerte → robust gegenüber Ausreißern und Schiefe.
- Kein Mittelwert-Test im engen Sinn, sondern ein Test auf **Lageverschiebung**.
- Bei kleinen Stichproben ($n < 30$) und schiefen Daten meist die ehrlichere Wahl als `t.test()`.

 Faustregel: erst Verteilung anschauen (Histogramm, QQ-Plot), dann Test auswählen — nicht umgekehrt.

Kreuztabellen: `chisq.test()` und `fisher.test()`

```
1 strom_klass <- strom_tn |>
2   mutate(preis_klasse = ifelse(strompreis > 0.15,
3                               "Spitze", "normal"))
4
5 tab <- table(strom_klass$phase, strom_klass$preis_klasse)
6 tab
7
8 #  $\chi^2$ -Test für unabhängige kategoriale Merkmale
9 chisq.test(tab)
10
11 # Fisher: exakter Test, sinnvoll bei
12 # kleinen erwarteten Häufigkeiten (< 5)
13 fisher.test(tab)
```

`chisq.test()`

Asymptotisch. Standardwerkzeug für Kreuztabellen mit ausreichender Zellbesetzung.

`fisher.test()`

Exakt. Vorzugswahl bei kleinen Stichproben und dünn besetzten Zellen.

Warnung beachten

`chisq.test()` warnt automatisch, wenn erwartete Häufigkeiten < 5 — dann `fisher.test()` nehmen.

p-Werte richtig lesen

Was ein p-Wert sagt

Wahrscheinlichkeit, **unter der Nullhypothese** ein mindestens so extremes Ergebnis zu beobachten. Nichts weiter.

Was er nicht sagt

Weder "die Nullhypothese ist falsch", noch "der Effekt ist groß", noch "das Ergebnis ist praktisch relevant".

Effektgröße zählt

Bei großen n ist fast alles "signifikant". Berichten Sie **Effektgröße + Konfidenzintervall**, nicht nur p.

Praxisregel

Statt " $p < 0.05 \rightarrow \text{ja/nein}$ " lieber: Punktschätzer, 95-%-KI, Effektgröße, dann p. So bleibt die Aussage robust.



Mehrfaches Testen ohne Korrektur (`p.adjust()`, Bonferroni, BH) treibt die Falsch-Positiv-Rate hoch. Immer dokumentieren, wie viele Tests gerechnet wurden.

Lineare Regression: `lm()`

```
1 modell <- lm(strompreis ~ co2_faktor,  
2             data = strompreise)  
3  
4 summary(modell)  
5 # Coefficients:  
6 #   (Intercept)   -0.0234  
7 #   co2_faktor    0.00046 ***  
8 # Multiple R-squared: ~0.55  
9  
10 # Punkt-Schätzer extrahieren  
11 coef(modell)  
12 confint(modell)      # 95%-KI je Koeffizient  
13 fitted(modell)       # Vorhersagen für die Daten  
14 residuals(modell)    # Residuen
```

- $y \sim x$ ist die **Formel-Schreibweise** — sie taucht in fast jedem Modellaufruf wieder auf.
- `summary(modell)` ist das Standard-Werkzeug: Koeffizienten, Standardfehler, t-Werte, p-Werte, R^2 , F-Test.
- R^2 beschreibt den Anteil erklärter Varianz; bei mehreren Prädiktoren lieber **Adjusted R^2** verwenden.
- Stets prüfen: ist der Zusammenhang **linear**?

Diagnostik: `plot(modell)` und `broom`

```
1 par(mfrow = c(2, 2))
2 plot(modell)           # 4 Diagnostik-Plots:
3                         # Residuals vs Fitted
4                         # Q-Q Residuals
5                         # Scale-Location
6                         # Residuals vs Leverage
7
8 # Aufgeräumt im tidyverse-Stil
9 library(broom)
10 tidy(modell)           # Koeffizienten als Tibble
11 glance(modell)         # ein Tibble mit R2, AIC, ...
12 augment(modell)       # Daten + Residuen + .fitted
```

Residuals vs Fitted

Sollte zufällig um 0 streuen — keine Trichter, keine Bögen.

Q-Q Plot

Punkte auf der Diagonalen → Residuen sind annähernd normal.

Leverage

Hebelpunkte ausmachen: einzelne Beobachtungen, die das Modell stark ziehen.

Multiple Regression und Interaktionen

```
1 # Mehrere Prädiktoren
2 m1 <- lm(strompreis ~ co2_faktor + emissionen,
3         data = strompreise)
4
5 # Interaktion mit Tag/Nacht: wirkt CO2 tagsüber anders?
6 m2 <- lm(strompreis ~ co2_faktor + emissionen +
7         co2_faktor:phase,
8         data = strom_tn)
9
10 # Modelle vergleichen
11 anova(m1, m2)           # F-Test der Modellverbesserung
12 AIC(m1, m2)            # niedriger = besser
```

- `+` fügt einen Prädiktor hinzu, `:` bildet eine Interaktion, `*` bildet **Haupteffekte + Interaktion**.
- **Kategoriale Prädiktoren** (Factors) erzeugen Dummy-Variablen; eine Stufe wird als **Reference-Level** weggelassen.
- Mit `relevel()` oder `forcats::fct_relevel()` lässt sich die Referenz gezielt setzen.
- `anova()` und `AIC()` sind die zwei üblichen Modellvergleiche.

Logistische Regression: `glm()`

```
1 # Binäres Ziel: Strompreis über 0.15 EUR (Spitze) ja/nein?
2 modell_log <- glm(
3   I(strompreis > 0.15) ~ co2_faktor + emissionen,
4   data = strompreise,
5   family = binomial
6 )
7
8 summary(modell_log)
9
10 # Odds Ratios – handlicher als Log-Odds
11 exp(coef(modell_log))
12 exp(confint(modell_log))
13
14 # Vorhersage als Wahrscheinlichkeit
15 predict(modell_log, newdata = neu,
16         type = "response")
```

- `family = binomial` schaltet von linearer auf **logistische** Regression um.
- Koeffizienten sind **Log-Odds**; via `exp()` werden Odds Ratios daraus.
- **OR > 1** → Merkmal erhöht die Wahrscheinlichkeit des Ziel-Ereignisses (hier: Preisspitze); **OR < 1** → senkt sie.
- `predict(..., type = "response")` liefert Wahrscheinlichkeiten zwischen 0 und 1.
- Typische Anwendungsfälle: Kauf, Kündigung, Bestanden/Durchgefallen, Risikoflags.

R-Statistik ohne Zusatzpakete

Statistik first class

`t.test`, `lm`, `glm`, `aov`, `prcomp` ohne externes Paket.

Formel-Syntax

`y ~ x1 + x2` ist über alle Modellklassen hinweg konsistent.

Diagnostik integriert

`plot(modell)` produziert sofort die wichtigsten vier Plots.

Modelle als Objekte

Ein Modell ist ein Objekt mit Methoden: `summary`, `predict`, `coef`, `confint`, `anova`.

Ökosystem

`broom`, `performance`, `marginaleffects`, `easystats` ergänzen ohne Bruch.

Konsequenz für den Kurs

Sie kennen mit `lm`, `glm`, `summary`, `plot` und `anova` schon das **Standard-Werkzeug** für 80 % der Analyse-Aufgaben.

Teil VII — Grafiken mit ggplot2

Diagramme als **strukturierte Deklaration** statt Klick-Folge.

Grammar of Graphics

Daten + Mapping +
Geometrie + Layer + Theme.

Drei Standardfälle

Balken, Streuung, Linie — mit
Bild und Lesart.

Erweiterungen

scales, patchwork, ggrepel,
plotly, gganimate.

Grammar of Graphics in fünf Bausteinen

1 — Daten

Welcher Tibble ist die Quelle? Tidy Data zählt sich hier doppelt aus.

2 — Mapping (`aes()`)

Welche Spalte landet auf x, y, Farbe, Form, Größe, Facet?

3 — Geometrie (`geom_*`)

Balken, Punkt, Linie, Boxplot, Histogramm, Smoother.

4 — Skalen + Koordinaten

Achsenformatierung, Farbskalen, Transformationen (`scale_y_log10`).

5 — Labels + Theme

`labs()` und `theme_*` machen aus Rohgrafik einen lesbaren Bericht.

Merksatz

Ein ggplot2-Diagramm **wächst additiv** — jedes `+` ist eine neue Schicht.

Warum ggplot2 statt Base R oder Excel?

Aspekt	Base R	Excel	ggplot2
Lernkurve	flach	sehr flach	mittel
Konsistenz über Diagrammtypen	mittel	gering	hoch
Reproduzierbarkeit	gegeben	gering	sehr hoch
Skalierbar auf viele Plots	aufwendig	sehr aufwendig	sehr gut (Faceting)
Publikationsreife	gut mit Aufwand	begrenzt	sehr gut
Interaktiv	nein	begrenzt	via <code>plotly::ggplotly()</code>

 **Konsequenz:** Für analytische Berichte ist ggplot2 die Standardroute. Excel-Charts haben Berechtigung in Ad-hoc-Kommunikation mit Fachbereich.

Balkendiagramm

```
1 strompreise |>
2   mutate(monat = lubridate::floor_date(zeit, "month")) |>
3   group_by(monat) |>
4   summarise(avg_strompreis = mean(strompreis),
5             .groups = "drop") |>
6   ggplot(aes(x = monat, y = avg_strompreis)) +
7   geom_col(fill = "#2459a6", width = 25) +
8   labs(title = "Mittlerer Strompreis pro Monat 2024",
9        x = "Monat", y = "Strompreis (EUR/kWh)") +
10  theme_minimal()
```

Lesart: Zwölf Balken zeigen den Monatsmittelwert; Spitzen liegen typischerweise im Winter.

Was zu sehen ist

- x-Achse: zwölf Monate von 2024
- y-Achse: Mittelwert von `strompreis`
- Einheitliche Farbe — kein Mapping nötig, weil nur eine Reihe

Streudiagramm

```
1 ggplot(strompreise,
2       aes(x = co2_faktor, y = strompreis)) +
3   geom_point(alpha = 0.25, size = 1.2,
4             color = "#2459a6") +
5   geom_smooth(method = "lm", se = FALSE,
6             color = "#d33") +
7   labs(title = "CO2-Faktor und Strompreis",
8        x = "CO2-Faktor (g/kWh)",
9        y = "Strompreis (EUR/kWh)") +
10  theme_minimal()
```

Lesart: Eine positive Tendenz ist erkennbar — höhere CO₂-Intensität geht mit höherem Strompreis einher. `geom_smooth(method = "lm")` legt die Regressionsgerade als Schicht darüber.

Was zu sehen ist

- 8 784 Punkte (eine pro Stunde 2024)
- `alpha = 0.25` macht Überlagerungen sichtbar
- rote Linie = lineare Regression

Linendiagramm

```
1 wetter <- read_csv("data/berlin_weather_2024.csv",
2                     show_col_types = FALSE) |>
3   mutate(time = lubridate::as_datetime(time))
4
5 wetter |>
6   mutate(tag = lubridate::as_date(time)) |>
7   group_by(tag) |>
8   summarise(t_mittel = mean(temperature_2m),
9             .groups = "drop") |>
10  ggplot(aes(x = tag, y = t_mittel)) +
11    geom_line(color = "#2459a6", linewidth = 0.6) +
12    labs(title = "Tagesmittel-Temperatur Berlin 2024",
13         x = "Datum", y = "Temperatur (°C)") +
14    theme_minimal()
```

Lesart: Eine Linie zeigt Niveau und zeitliche Muster zugleich. Aggregation auf Tagesebene reduziert das Rauschen der Stundenwerte.

Was zu sehen ist

- 366 Tagespunkte (2024 = Schaltjahr)
- klarer Jahresgang: Winter unten, Sommer oben
- `as_datetime()` vermeidet die Stolperfalle "String statt Datum"

Boxplot und Histogramm

```
1 # Boxplot: Verteilung pro Gruppe
2 strompreise |>
3   mutate(monat = lubridate::month(zeit,
4                                     label = TRUE)) |>
5   ggplot(aes(x = monat, y = strompreis,
6               fill = monat)) +
7   geom_boxplot(alpha = 0.7, show.legend = FALSE) +
8   labs(title = "Strompreisverteilung pro Monat") +
9   theme_minimal()
```

Boxplot

Zeigt Median, IQR, Whisker, Ausreißer — perfekt für Gruppenvergleich.

```
1 # Histogramm: Verteilung einer Variablen
2 ggplot(strompreise, aes(x = strompreis)) +
3   geom_histogram(bins = 40,
4                   fill = "#2459a6",
5                   color = "white") +
6   labs(title = "Verteilung der Strompreise 2024",
7         x = "Strompreis (EUR/kWh)",
8         y = "Anzahl Stunden") +
9   theme_minimal()
```

Histogramm

Zeigt die Verteilungsform. `bins` bewusst wählen — zu wenig glättet weg, zu viel zerfasert.

Facets, Skalen und Themes

```
1 strompreise |>
2   mutate(quarteral = lubridate::quarter(zeit,
3                                     with_year = FALSE)) |>
4   ggplot(aes(x = co2_faktor, y = strompreis,
5             color = factor(quarteral))) +
6   geom_point(alpha = 0.4, size = 0.8) +
7   facet_wrap(~ quarteral, labeller = label_both) +
8   scale_y_continuous(labels = scales::label_number(suffix = " EUR")) +
9   scale_color_brewer(palette = "Set2", name = "Quarteral") +
10  labs(title = "CO2-Faktor und Strompreis je Quartal",
11       subtitle = "Gleiche Struktur, getrennte Panels") +
12  theme_minimal(base_size = 12)
```

facet_wrap()

Ein Panel je Gruppe — ideal, um Strukturen über Kategorien zu vergleichen.

scale_*()

Steuert Achsen-Formatierung, Transformationen, Farb- und Größenmapping.

theme_*()

`theme_minimal`, `theme_bw`, `theme_classic` — und feinjustierbar mit `theme(...)`.

Reihenfolge zählt

Spätere Layer überschreiben frühere — `theme()` immer ans Ende.

ggplot2-Erweiterungspakete

scales

Achsen formatieren: Währung, Prozent, Tausendertrenner, Log-Achsen.

patchwork

Plots kombinieren mit `p1 + p2 / p3` — kein `grid.arrange`-Boilerplate mehr.

ggrepel

`geom_text_repel()` setzt Labels ohne Überlappung.

ggthemes

Vorgefertigte Themes: Economist, FiveThirtyEight, Tufte, Stata, Solarized.

plotly::ggplotly()

Macht aus jedem ggplot ein interaktives HTML-Widget — Tooltip + Zoom.

gganimate

Animierte Plots mit `transition_time()` — für Demos und Konferenzen, selten für Berichte.

```
1 # Mini-Beispiel scales + patchwork
2 library(scales); library(patchwork)
3 p1 <- ggplot(strompreise, aes(co2_faktor, strompreis)) + geom_point(alpha = 0.2) +
4     scale_y_continuous(labels = label_currency(prefix = "", suffix = " €"))
5 p2 <- ggplot(strompreise, aes(strompreis)) + geom_histogram(bins = 40)
6 p1 + p2
```


Was zeichnet ein gutes Diagramm aus?

Klare Aussage

Titel sagt, **was** gezeigt wird; Untertitel sagt, **warum es interessant** ist.

Saubere Achsen

Einheiten, sinnvolle Skalierung, lesbare Beschriftungen.

Farbe mit Funktion

Farben kodieren Gruppen oder Werte — nicht Dekoration.

Weniger ist mehr

Keine 3D-Effekte, keine Regenbogen-Paletten, keine doppelten Y-Achsen.



Analytische Regel: Ein Diagramm hat einen Job — eine konkrete fachliche Frage **präzise zu beantworten**. Alles andere ist Dekoration.

Teil VIII — Berichte mit Quarto und Markdown

Eine Quelle, mehrere Formate, reproduzierbar.

Quellstruktur

YAML, Markdown-Text,
Code-Chunks.

Formate

HTML, PDF, Word, Revealjs,
Beamer aus einer Datei.

Interaktiv

Tabellen mit DT/reactable,
Diagramme mit plotly.

Warum Quarto in einem R-Kurs?

Code + Text + Ergebnis

Analyse und Erklärung in einer Datei — keine “wo war noch der Plot?”-Diskussion.

Wiederholbar

Mit neuen Daten einmal rendern, der Bericht aktualisiert sich vollständig.

Eine Quelle, viele Formate

HTML für den Browser, PDF für Print, Word für Stakeholder, Revealjs für Slides.

Sprachunabhängig

Quarto kennt R, Python, Julia, Observable — derselbe Workflow für gemischte Teams.

Quarto vs. R Markdown

Aspekt	R Markdown	Quarto
Engine	knitr (R-zentriert)	knitr + jupyter (multi-language)
Output-Formate	viele über <code>rmarkdown</code>	noch mehr , vereinheitlicht
YAML-Optionen	unterschiedlich pro Format	einheitliche Optionen quer durch Formate
Wartung	weiter unterstützt, Posit setzt auf Quarto	aktive Weiterentwicklung
Slides	<code>ioslides</code> , <code>xaringan</code>	<code>revealjs</code> als Standard

 Quarto ist der **Nachfolger** von R Markdown. Bestehende `.Rmd` lassen sich in vielen Fällen direkt zu `.qmd` umbenennen und rendern.

YAML-Header

```
1 ---
2 title: "Strompreis-Analyse 2024"
3 subtitle: "Monatliche Kennzahlen aus Stundendaten"
4 author: "Dr. Devadze"
5 date: today
6 format:
7   html:
8     toc: true
9     toc-depth: 2
10    code-fold: true
11    theme: cosmo
12    embed-resources: true
13  pdf:
14    documentclass: article
15    geometry: margin=2.5cm
16 execute:
17   echo: false
18   warning: false
19 ---
```

- `toc` baut automatisch ein Inhaltsverzeichnis.
- `code-fold: true` versteckt Code, der Leser kann ihn aufklappen.
- `embed-resources: true` macht ein **stand-alone** HTML (keine Asset-Ordner).
- `execute`: setzt globale Defaults für alle Chunks.
- Mehrere Formate gleichzeitig: `quarto render bericht.qmd --to html,pdf`.

Markdown und Code-Chunks

Überblick

Der Datensatz enthält **8 784 stündliche Beobachtungen** für ganz 2024.

```
```${r}
#| label: deskriptiv
#| echo: false
#| warning: false

library(dplyr); library(lubridate)
strompreise |>
 mutate(monat = floor_date(zeit, "month")) |>
 group_by(monat) |>
 summarise(avg_preis = mean(strompreis))
```
```

Markdown-Basics

Heading · ****fett**** · **kursiv** · [Link](url) · -
Liste

Chunk-Optionen

echo, eval, output, warning, message, cache,
fig-width, fig-cap.

Tipp

Optionen modern mit `#|` direkt im Chunk schreiben —
leichter zu lesen als die alte `{r echo=FALSE}`-Form.

Output-Formate im Überblick

HTML

Standard für Berichte. Mit `embed-resources: true` eine einzige Datei zum Verschicken.

PDF

Über LaTeX. Vorteil: druckreif. Nachteil: LaTeX-Installation nötig (`tinytex::install_tinytex()`).

Word (docx)

Für Stakeholder, die im Word weiterarbeiten. Templates per `reference-doc:` einbindbar.

Revealjs

HTML-Slides — genau das, was Sie gerade sehen.

Beamer

PDF-Slides über LaTeX, klassisch in akademischem Umfeld.

Empfehlung

Im Alltag: HTML als Standard, Word für Übergabe, PDF nur wenn Druck wirklich gebraucht wird.

Interaktive Tabellen

```
1 # 1) gt: schöne statische Tabellen (HTML, Word, PDF)
2 library(gt); library(lubridate)
3 strompreise |>
4   mutate(monat = floor_date(zeit, "month")) |>
5   group_by(monat) |>
6   summarise(avg_preis = mean(strompreis)) |>
7   gt() |>
8   fmt_currency(columns = avg_preis, currency = "EUR") |>
9   tab_header(title = "Mittlerer Strompreis pro Monat")
10
11 # 2) DT: interaktiv mit Sortieren + Suchen
12 library(DT)
13 datatable(strompreise, filter = "top",
14           options = list(pageLength = 10))
15
16 # 3) reactable: moderner, anpassbarer
17 library(reactable)
18 reactable(strompreise, searchable = TRUE,
19           defaultPageSize = 10)
```

gt

Für **statische, hochwertig formatierte** Tabellen in HTML/PDF/Word.

DT

Wrapper um DataTables.js. Klassiker für interaktive HTML-Tabellen.

reactable

Moderner Nachfolger — schöner, mehr Anpassung, gut für Dashboards.

Interaktive Diagramme:

plotly::ggplotly()

```
1 library(plotly)
2
3 p <- ggplot(strompreise,
4             aes(x = co2_faktor, y = strompreis,
5                 text = paste("Zeit:", zeit,
6                               "<br>Preis:", strompreis))) +
7   geom_point(alpha = 0.3, size = 1.2,
8             color = "#2459a6") +
9   theme_minimal()
10
11 # Aus ggplot ein interaktives Widget
12 ggplotly(p, tooltip = "text")
```

- `ggplotly()` macht aus jedem ggplot ein **interaktives HTML-Widget**: Hover-Tooltip, Zoom, Pan, Reset.
- Nur in **HTML-Outputs** sichtbar — im PDF wird das statische ggplot eingebettet.
- Für Dashboards: `htmlwidgets` als gemeinsamer Standard (plotly, DT, reactable, leaflet, networkD3 ...).

Reproduzierbarer Workflow

1

`data/`

Rohdaten read-only ablegen, nie überschreiben.

2

`R/analyse.R`

Aufbereitung, Modelle, Kennzahlen — als reines R-Skript, gut versionierbar.

3

`figures/`

Plots als PNG/PDF speichern, falls im Bericht referenziert.

4

`bericht.qmd`

Quarto-Datei lädt das Skript via `source()` und kommentiert Ergebnisse fachlich.

5

`quarto render`

Aus Terminal oder RStudio. Output landet konsistent in `_site/` oder daneben.

Bonus

Mit `renv::init()` + `renv::snapshot()` Pakete versionieren — voll reproduzierbar auch nach 12 Monaten.

```
1 quarto render bericht.qmd --to html
2 quarto render bericht.qmd --to docx
```

Teil IX — KI in R und RStudio

Werkzeuge, die heute schon Zeit sparen — und die Stellen, an denen man vorsichtig sein muss.

In RStudio

Copilot, chattr, gptstudio als Add-ins.

Aus R-Code

`ellmer` für Cloud-LLMs,
`rollama` für lokale Modelle.

On-Prem first

Datenschutz und
Lizenzfragen ehrlich
mitdenken.

KI im R-Workflow: Nutzen und Risiken

Wo KI heute hilft

- Boilerplate-Code (ggplot-Skelette, Test-Stubs)
- Code-Erklärungen und Reviews
- Erste Berichts-Entwürfe
- Daten-Exploration als Sparringspartner

Wo KI noch nicht hilft

- Domänenspezifische Statistik-Methodik
- Verständnis Ihrer **konkreten** Geschäftslogik
- Verlässliche Quellenangaben
- Garantiert korrekte Modellauswahl

Risiken

- **Halluzinationen** (erfundene Funktionsnamen, Pakete)
- **Datenleck** (Spalten, Identifier, Geschäftsdaten an Cloud-API)
- **Lizenz** (Copilot-Code-Snippets, Lernkorpus, IP-Fragen)

Goldene Regel

KI generiert Vorschläge. **Verantwortung bleibt beim Menschen** — jeden Vorschlag prüfen und mit kleinem Test absichern.

GitHub Copilot in RStudio

Setup (kurz)

1. GitHub-Account mit Copilot-Abo
2. RStudio → **Tools** → **Global Options** → **Copilot**
3. **Sign in** über GitHub-Token
4. Pro Datei aktivieren oder global

Was Copilot gut kann

- ggplot-Skelette ergänzen
- repetitive dplyr-Ketten
- Test-Stubs und Roxygen-Kommentare
- Boilerplate für Funktionen

Was Copilot nicht zuverlässig kann

- Branchenspezifische Statistik-Auswahl
- Korrekte Behandlung Ihrer Edge Cases
- Verlässliche Paketwahl bei Spezial-Themen

Vorsicht

Code-Vorschläge entstehen aus Trainingsdaten. Bei sensiblen Projekten: - Lizenzkonformität prüfen - keine Geschäftsgeheimnisse als Kommentar "fragen" - Tests **immer** selber schreiben

e1lmer: Idiomatic LLM Access from R

```
1 library(e1lmer)
2
3 # Chat-Objekt für Anthropic, OpenAI, Google, ...
4 chat <- chat_anthropic(
5   model = "claude-sonnet-4-5",
6   system_prompt = "Sie sind Statistik-Assistent."
7 )
8
9 # Einfache Frage
10 chat$chat("Erkläre Spearman-Korrelation in zwei Sätzen.")
11
12 # Strukturierte Ausgabe via Schema
13 type_befund <- type_object(
14   effekt = type_string(),
15   p_wert = type_number(),
16   hinweis = type_string()
17 )
18 chat$chat_structured(
19   "Bewerte: r = 0.42, p = 0.001, n = 80",
20   type = type_befund
21 )
```

- e1lmer ist das **moderne, von Posit gepflegte** LLM-Paket für R.
- Provider-agnostisch: Anthropic, OpenAI, Google, **Ollama (lokal)**, ...
- **Structured output** über Typen-Schema — kein JSON-Parsing von Hand.
- Streaming, Tool-Use, Vision: alles vorhanden.
- Reife Alternative zu älteren Wrappern.

Lokale LLMs mit `rollama` / `ollama-r`

```
1 # Voraussetzung: Ollama läuft lokal (ollama.com)
2 # z. B.      ollama pull llama3.1
3 #           ollama pull qwen2.5
4
5 library(rollama)
6
7 # Modell-Check
8 ping_ollama()
9 list_models()
10
11 # Einfache Abfrage
12 query("Erkläre einen  $x^2$ -Test in zwei Sätzen.",
13       model = "llama3.1")
14
15 # Auch via ellmer mit Ollama-Backend
16 chat <- ellmer::chat_ollama(model = "llama3.1")
17 chat$chat("Welcher R-Test passt zu zwei Mittelwerten?")
```

Was Ollama liefert

Lokaler LLM-Server, OpenAI-kompatible API. Kein Account, keine Cloud, **keine Datenabwanderung**.

Wann lokal Pflicht ist

Personenbezogene Daten, Kunden- und Vertragsdaten, Forschungs- und Patientendaten — alles unter Vertraulichkeit.

Praxistipp

On-prem-LLMs laufen heute schon mit 7B/8B-Modellen auf Notebooks.

chattr und gptstudio

chattr (Posit)

- Sidebar-Chat direkt in RStudio
- Backends: OpenAI, Copilot Chat, **Ollama (lokal)**, LlamaGPT
- Sehr leichtgewichtig
- Setup: `chattr::chattr_app()` öffnen

```
1 install.packages("chattr")
2 library(chattr)
3 chattr_use("ollama")    # oder "openai"
4 chattr_app()
```

gptstudio

RStudio-Add-ins: - "Spelling and Grammar" - "Code Review" -
"Inline-Erklärung" - "Custom-Prompt"

Backends: OpenAI, Anthropic, HuggingFace, **Ollama**, Google.

```
1 install.packages("gptstudio")
2 # Add-ins erscheinen im Addins-Menü
```


Welcher Workflow wofür?

Sensible Daten

- **lokal** via `rollama` oder `chattr` + Ollama
- niemals Spaltennamen oder Werte an Cloud-LLMs
- on-prem-Modelle reichen für 80 % der Code-Hilfen

Boilerplate / ggplot-Skelette

- Copilot ist schnell, aber prüfen
- `gptstudio` "Code Review" als zweites Augenpaar
- bei Lizenzsorgen → lokal mit Code-Modell (z. B. Qwen-Coder)

Erste Berichts-Entwürfe

- `ellmer` mit Schema → strukturierter Output
- Quarto-Vorlagen mit LLM-Befunden befüllen
- **nie** Roh-Ergebnisse ungeprüft veröffentlichen

Goldene Regeln

1. Keine vertraulichen Spalten/Werte in Cloud-Prompts.
2. Jeden Vorschlag mit kleinem Test absichern.
3. Quellen und Pakete eigenständig überprüfen — Halluzinationen kommen vor.

Wo KI im R-Workflow Zeit spart

ggplot-Skelette

“Bau mir einen facettierten Boxplot mit Theme minimal” — Copilot oder `gptstudio` Add-in.

SQL → dplyr

Vorhandene SQL-Abfragen in `dbplyr` / `dplyr` umschreiben lassen — meist 80 % korrekt.

Test-Stubs

Aus Funktion mit Roxygen-Doku `testthat`-Tests vorschlagen lassen.

Code-Review

“Welche Edge Cases fehlen?” als Sparringspartner — selten neu, aber hilft beim Übersehen.

Berichtstext

Aus `broom::tidy()`-Tabelle einen Befund-Satz formulieren lassen — Sie prüfen Zahlen, KI prüft Sprache.

Dokumentation

Funktion → Roxygen-Block. Spart Routine, ersetzt aber kein fachliches Beispiel.

Mini-Übung 2: Statistik und Bericht

Aufgabe

Datei: `labs/lab2_statistik_und_bericht.R`

1. `data/strompreise.csv` einlesen und Spalten in `zeit`, `co2_faktor`, `strompreis`, `emissionen` umbenennen.
2. Lineare Regression `strompreis ~ co2_faktor` rechnen.
3. `summary(modell)` lesen und Koeffizienten + R^2 notieren.
4. Streudiagramm mit Regressionsgerade (`geom_smooth(method = "lm")`).
5. Einen kleinen Quarto-Bericht `bericht.qmd` aufsetzen, der Modell und Plot zeigt.
6. Als HTML rendern: `quarto render bericht.qmd`.

Bonus

- `glm()` mit `family = binomial` für ein binäres Merkmal (z. B. `strompreis > 0.15` als Ziel).
- Plot interaktiv machen mit `ggplotly()`.
- Bericht zusätzlich als Word rendern.

Lernziel

Einen kompletten Mini-Workflow von Daten → Modell → Plot → Bericht **selbst durchspielen**.

Kernaussagen

- **Statistik in R:** `lm`, `glm`, `t.test`, `cor.test`, `chisq.test` decken bereits den Großteil des Alltags ab. `broom` macht Output tidyverse-kompatibel.
- **p-Werte:** immer mit **Effektgröße + Konfidenzintervall** berichten — nie p allein.
- **ggplot2:** Daten → Mapping → Geometrie → Skalen → Theme. Drei Standardfälle (Balken, Scatter, Linie) decken die meisten Berichte ab.
- **Erweiterungen** (`scales`, `patchwork`, `ggrepel`, `plotly`) heben Berichte auf Publikationsniveau.
- **Quarto** verbindet Code, Text und Ergebnis in einer Quelle und rendert in HTML/PDF/Word/Slides. Mit `gt`, `DT`, `reactable`, `plotly` werden Berichte interaktiv.
- **KI im R-Workflow:** Copilot/ellmer/rollama sind echte Zeitgewinne — sensible Daten gehören lokal, jede Antwort wird geprüft.

Was lernt man nach diesem Kurs weiter?

Shiny

Interaktive Web-Apps in R. Natürliche Fortsetzung nach Quarto und plotly.

Fortgeschrittene Statistik

Mixed Models (`lme4`), Survival Analysis (`survival`), Bayesian (`brms` , `rstanarm`).

MLOps in R

`tidymodels` , `vetiver` für Deployment, `pins` für Modell-Registry.

R für Produktion

`plumber` für APIs, `targets` für Pipelines, `renv` für reproduzierbare Umgebungen.

Datenarbeit at Scale

`data.table` , `arrow` , `duckdb` für große Datenmengen.

Empfohlene Reihenfolge

1. Festigen (eigene Daten in den Workflow holen)
2. Shiny oder tidymodels (je nach Bedarf)
3. Produktion (renv, targets, Quarto-CI)

Quellen und Vertiefung

R for Data Science (R4DS)

r4ds.hadley.nz — kostenlos, der De-facto-Standard.

Posit Cheat Sheets

posit.co/resources/cheatsheets — `dplyr`, `ggplot2`, `purrr`, Quarto.

Quarto-Dokumentation

quarto.org — Referenz, Guides, Beispiele.

ModernDive

moderndive.com — Statistik + R für Einsteiger, sehr didaktisch.

Tidy Modeling with R

tmwr.org — `tidymodels` -Buch, frei online.

KI-Pakete

- `ellmer` : ellmer.tidyverse.org
- `rollama` : jbgruber.github.io/rollama
- `chattr` : mlverse.github.io/chattr
- `gptstudio` : michelnivard.github.io/gptstudio